

PlayBook / 03-Patterns / Repository.md

Repository Pattern & Abstracción de Datos

1. ¿Por qué el Repository Pattern?

Abstrae la lógica de persistencia del resto de la aplicación, permitiendo que la capa de dominio/aplicación interactúe con colecciones como si estuvieran en memoria.

2. Implementación C#

```
public interface ICustomerRepository : IGenericRepository<Customer>
{
    Task<Customer?> GetCustomerWithOrdersAsync(Guid customerId);
}
```

Simulated Interview Response (English - Senior / Staff Level)

Interviewer: "Since Entity Framework Core already implements a Repository with DbSet, why would you wrap it with a custom Repository Pattern?"

Your Answer:

"That's a classic discussion in .NET architecture. While it's true that DbContext acts as a Unit of Work and DbSet acts as a Repository, introducing a custom repository layer in Clean Architecture serves a distinct purpose: it completely decouples the Domain and Application layers from EF Core primitives. This prevents database leaky abstractions—such as directly leaking IQueryable to controllers—and allows pure unit testing without spinning up an in-memory database. However, for read-heavy operations or simple CQRS queries, I often bypass custom repositories and query directly using EF Core or Dapper for optimal performance."